ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: CHUN-WU LIU**
- **Chinese Name:劉俊吾**
- **Email:a4264428@gmail.com**
- **Student ID:115062506**
- **University: National Tsing Hua University**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. The sci.med category contains 594 documents.

2. Frequent pattern mining treats each document as a transaction and keeps only the word sets whose support reaches the minimum support threshold. Isolating a single category corresponds to the KDD step of creating a task-relevant target dataset. If documents from other categories are mixed in, two problems arise: (a) the denominator grows, so the support of topic-specific word sets is diluted and may fall below the threshold, causing them to be missed; (b) the surviving patterns are dominated by generic, cross-topic words (e.g., "subject", "organization") that cannot be attributed to any specific topic.

AI used for this answer: Yes

If yes, how: After completing my answers myself, I used AI to translate them into English and polish the grammar and wording.

External references (links):

- Course slides, Yi-Shin Chen, *Data Mining*, "Session 1 and 2 – Introduction Handout", p. 7 "KDD Processes": "Pre-processing … Creating a target data set: data selection"
- Same handout, p. 8 "Knowledge Discovery (KDD) Process": selecting "Task-relevant Data" from databases / data warehouse before data mining
- Same handout, p. 77 "Selection": "Selection may also involve choosing a subset of objects"
- J. Han, M. Kamber, J. Pei, *Data Mining: Concepts and Techniques* (3rd ed.), Chapter 6 slides, p. 7 "Basic Concepts: k-Itemsets and Their Supports": (relative) support is "the fraction of transactions that contains X" — https://hanj.cs.illinois.edu/cs412/bk3_slides/06FPBasic.pdf

---

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer:

1. By default, dropna() removes an entire row if any single field is missing. These 40% of documents would therefore be discarded in full even though only their labels are missing, losing nearly half of the usable text. Unsupervised analyses such as the document-term matrix, term-frequency statistics, and frequent pattern mining do not need labels at all, yet they would lose these documents as well. Moreover, if the labels are not missing at random (e.g., one category is more likely to be unlabeled), the remaining class proportions no longer represent the original data, which biases the analysis.

2. An objective alternative is to infer the missing values from the available data. For example, use the 60% of labeled documents as a training set, convert the text into a document-term matrix, train a classifier (e.g., Naive Bayes or a decision tree), and predict the labels of the remaining documents, i.e., infer the most probable value from the data.

AI used for this answer: Yes

If yes, how: After completing my answers myself, I used AI to translate them into English and polish the grammar and wording.

External references (links):

- Course slides, Yi-Shin Chen, *Data Mining*, "Session 1 and 2 – Introduction Handout", p. 32 "Missing Values": "Handling missing values: Eliminate Data Objects / Estimate Missing Values / Ignore the Missing Value During Analysis / Replace with all possible values"
- Course slides, Yi-Shin Chen, *Data Mining*, "Session 3 and 4 – Classification Handout", p. 3 "Classification: Definition": "Given a collection of records (training set) … One of the attributes is the class … Find a model for class attribute … previously unseen records should be assigned a class as accurately as possible"
- Same handout, p. 4 "Supervised vs. Unsupervised Learning": "Unsupervised learning: The class labels of training data is unknown"
- Same handout, p. 48 "Naïve Bayes Classifier"; decision trees on p. 8–10 "Decision Tree"
- pandas documentation, DataFrame.dropna (default how='any': "If any NA values are present, drop that row or column") — https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html
- J. Han, M. Kamber, J. Pei, *Data Mining: Concepts and Techniques* (3rd ed.), Chapter 3 slides, p. 9 "How to Handle Missing Data?": "Ignore the tuple: usually done when class label is missing … not effective when the % of missing values per attribute varies considerably"; "Fill in it automatically with … the most probable value: inference-based such as Bayesian formula or decision tree" — https://hanj.cs.illinois.edu/cs412/bk3_slides/03Preprocessing.pdf
- scikit-learn documentation, "Semi-supervised learning": "a situation in which in your training data some of the samples are not labeled"; self-training "predicts labels for

the unlabeled samples and adds a subset of these labels to the labeled dataset" — https://scikit-learn.org/stable/modules/semi_supervised.html

---

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. No. After aligning the counts by category name, comp.graphics rises from 25.88% to 27.80% (+1.92 percentage points) and soc.religion.christian falls from 26.54% to 24.80% (−1.74 percentage points), while sci.med and alt.atheism differ by less than 0.4 percentage points. The ranking also changes: soc.religion.christian is the largest category in the original data, but comp.graphics is the largest in the sample. In addition, the notebook's side-by-side bar chart plots each value_counts() result in its own order; because the two orders differ, the orange bars are misaligned with the category labels, so a table comparison is needed to see the true differences.

2. Simple random sampling only guarantees that every record has the same probability of being selected; it does not guarantee that class proportions are preserved. Even this fairly balanced dataset (21%–27% per class) drifted by nearly 2 percentage points. In a highly imbalanced dataset, the expected sample size of a minority class is small and its relative fluctuation is large, so the class may be severely under-represented or missed entirely (e.g., a class that makes up 0.1% of the data has an expected count of only 1 in a sample of 1,000 and about a 37% chance of not appearing at all). The sample then loses representativeness, and downstream analyses or models will ignore the minority class. Stratified sampling, which samples each class according to its proportion, avoids this problem.

AI used for this answer: Yes

If yes, how: After completing my answers myself, I used AI to translate them into English and polish the grammar and wording.

External references (links):

- Course slides, Yi-Shin Chen, *Data Mining*, "Session 1 and 2 – Introduction Handout", p. 39 "Key Principle For Effective Sampling": "The sample is representative … The approximately the same property as the original set of data"
- Same handout, p. 40 "Sample Size Matters (Contd.)": "What sample size is necessary to get at least one object from each of 10 groups"
- J. Han, M. Kamber, J. Pei, *Data Mining: Concepts and Techniques* (3rd ed.), Chapter 3 slides, p. 34 "Sampling": "Simple random sampling may have very poor performance in the presence of skew" — https://hanj.cs.illinois.edu/cs412/bk3_slides/03Preprocessing.pdf

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. It increases. X_counts grows from (2257, 35,482) to (2257, 35,788), adding 306 features.

2. Sparsity decreases (the matrix becomes denser), from 99.677% to 99.547%. Because stop words appear in almost every document, adding them back increases the number of columns by less than 1%, but each of these columns is almost completely filled, so the number of non-zero entries rises sharply and the proportion of zeros drops.

AI used for this answer: Yes

If yes, how: After completing my answers myself, I used AI to translate them into English and polish the grammar and wording.

External references (links):

- Course slides, Yi-Shin Chen, *Data Mining*, "Session 1 and 2 – Introduction Handout", p. 18 "Characteristics of Data Sets": "Sparsity: Only presence counts"

- Same handout, p. 21 "Document Data": "Each document can be converted as a term vector. Each term is a component (attribute) of the vector. The value of each component is the number of times the corresponding term in the document"
- Lab slides, "DM2026 Lab 1 Master Notebook slides", p. 13 "5.0 Introduction to Vectors and Matrices": "Most documents use only a tiny fraction of the global vocabulary. The resulting matrix consists almost entirely of zeros (often >98% sparse)"; p. 17 "Stop Word Removal": "stop_words='english' … Discards extremely common functional words"
- scikit-learn documentation, sklearn.feature_extraction.text.CountVectorizer (stop_words, default token_pattern) — https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

---

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold	Number of terms kept (keep == True)
0.0	35,482
0.01	5,089
0.05	1,491

1. The vocabulary shrinks sharply as the threshold increases. At 0.0, all 35,482 terms are kept, because no term has exactly the same count in every document, so every variance is greater than 0. At 0.01, only 5,089 terms remain (85.7% removed), and at 0.05 only 1,491 remain (95.8% removed).

2. Terms with very low document frequency and low counts are eliminated first. These are mostly names, misspellings, or identifiers (e.g., zrz, zubkoff, 4202).

3. A feature with near-zero variance has almost the same value (almost always 0) in nearly every document, so it provides no information for distinguishing documents or classes; it is an irrelevant feature and can be removed. Words that appear in only one or two documents also cannot generalize to new documents, and keeping them makes overfitting more likely.

AI used for this answer: Yes

If yes, how: After completing my answers myself, I used AI to translate them into English and polish the grammar and wording.

External references (links):

- Course slides, Yi-Shin Chen, *Data Mining*, "Session 1 and 2 – Introduction Handout", p. 43 "Dimensionality Reduction": "Purpose: Avoid curse of dimensionality / Reduce amount of time and memory required by data mining algorithms … May help to eliminate irrelevant features or reduce noise"
- Same handout, p. 46 "Feature Subset Selection": "Another way to reduce dimensionality of data … Irrelevant features: Contain no information that is useful for the data mining task at hand"
- scikit-learn documentation, VarianceThreshold: "Feature selector that removes all low-variance features. This feature selection algorithm looks only at the features (X), not the desired outputs (y), and can thus be used for unsupervised learning." — https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.VarianceThreshold.html

---

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1. Neither list contains stop words. The Spearman top 10 is almost entirely graphics terminology (graphics, files, image, 3d, program, file, windows). The Pearson top 10 mixes domain terms (vga, video, card) with off-topic words, including words typical of help-request posts (thanks, hi, looking) and email header words (keywords, host). Both lists also contain negatively correlated religious words (people, god), which indicate "not graphics."

2. Pearson is computed on the raw counts, r = cov(X, Y) / (σX σY), and measures the linear relationship between two variables. The magnitude of every value affects the result, so it is sensitive to extreme values; when Y is 0/1, it is proportional to the difference between the two groups' mean counts divided by the overall standard deviation. Spearman first converts X and Y into ranks (ties receive the average rank) and then computes Pearson on those ranks, so it measures only the monotonic relationship, i.e., whether larger X tends to go with larger Y, without requiring linearity and without being affected by extreme values.

3. A word has a high Spearman but a low Pearson correlation when it appears consistently in the target category and rarely in the others (so the rank ordering is consistent), but its counts are highly skewed, with a few documents containing it an extremely large number of times.

AI used for this answer: Yes

If yes, how: After completing my answers myself, I used AI to translate them into English and polish the grammar and wording.

External references (links):

- Course slides, Yi-Shin Chen, *Data Mining*, "Session 1 and 2 – Introduction Handout", p. 60 "Correlation": "Correlation measures the linear relationship between objects … we standardize data objects, p and q, and then take their dot product"
- SciPy documentation, scipy.stats.pearsonr — https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html
- SciPy documentation, scipy.stats.spearmanr — https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

Both increase substantially. For alt.atheism, MaxFPGrowth finds 710 maximal patterns at minSup = 9 and 9,547 at minSup = 3 (about 13 times as many); the runtime also becomes noticeably longer, and the longest pattern grows from 9 to 69 words. Lowering the threshold lets many long-tail words that appear in only a few documents qualify, and the number of possible word combinations grows exponentially with the number of qualifying words, so both the FP-tree and the recursive search branches expand.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

minSup is an absolute threshold: each pattern is judged independently on whether its support is ≥ minSup, so the minimum quality is fixed in advance but the number of output patterns depends on the data. k is a relative ranking: all patterns are ranked by support and only the top k are kept, so the output size is fixed in advance but the minimum support depends on the data. For example, the lowest support in the result is 21 at k = 100 and drops to 16 at k = 500, which means k effectively sets the threshold automatically.

Answer (3) — Maximal vs standard frequent patterns:

Every subset of a frequent pattern is also frequent, so standard frequent pattern mining lists every subset of each long pattern; a frequent pattern with L words implies 2^L − 1 frequent sub-patterns, which creates massive redundancy. A maximal pattern is kept only if adding any further word would make it infrequent, so only the outermost patterns remain. Every other frequent pattern is a subset of some maximal pattern and can be derived from it, so it does not need to be listed.

AI used for this answer: Yes

If yes, how: After completing my answers myself, I used AI to translate them into English and polish the grammar and wording.

External references (links):

- J. Han, M. Kamber, J. Pei, *Data Mining: Concepts and Techniques* (3rd ed.), Chapter 6 slides, p. 11 "Challenge: There Are Too Many Frequent Patterns!": "A long pattern contains a combinatorial number of sub-patterns" — https://hanj.cs.illinois.edu/cs412/bk3_slides/06FPBasic.pdf
- Same slides, p. 13 "Expressing Patterns in Compressed Form: Max-Patterns": "A pattern X is a max-pattern if X is frequent and there exists no frequent super-pattern $Y \supset X$ … Max-pattern is a lossy compression!"
- Same slides, p. 16 "The Downward Closure Property of Frequent Patterns": "Apriori: Any subset of a frequent itemset must be frequent"
- PAMI documentation and examples (UdayLab): FAE (Top-K) — https://colab.research.google.com/github/UdayLab/PAMI/blob/main/notebooks/frequentPattern/topk/FAE.ipynb; MaxFPGrowth — https://colab.research.google.com/github/UdayLab/PAMI/blob/main/notebooks/frequentPattern/maximal/MaxFPGrowth.ipynb

---

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. Each point on a scatter plot needs two (or three) coordinates, but in X_counts each document is a point in a 35,482-dimensional space, where every word is an axis. A screen can display only two dimensions and humans can perceive at most three, so 35,482 axes cannot be drawn. If we simply picked two words as the x and y axes, most documents would overlap at the origin because about 99.7% of the matrix is zero, and the information in the remaining 35,480 dimensions would be discarded. Moreover, in such a high-dimensional, sparse space, the distances between points become nearly equal (the curse of dimensionality), so the very idea that documents of the same category lie close together becomes unreliable.

2. A dimensionality reduction algorithm must be applied first, transforming the high-dimensional data into two or three dimensions while preserving the original structure as much as possible. The helper file visualization_helpers.py uses PCA, t-SNE, and UMAP (e.g., compute_dimred_2d_for_filters and compute_dimred_3d).

AI used for this answer: Yes

If yes, how: After completing my answers myself, I used AI to translate them into English and polish the grammar and wording.

External references (links):

- Course slides, Yi-Shin Chen, *Data Mining*, "Session 1 and 2 – Introduction Handout", p. 42 "Curse of Dimensionality": "When dimensionality increases, data becomes increasingly sparse in the space that it occupies. Definitions of density and distance between points … become less meaningful"
- Same handout, p. 43 "Dimensionality Reduction": "Purpose: Avoid curse of dimensionality … Allow data to be more easily visualized … Techniques: Linear: Principle Component Analysis (PCA) … Others: supervised and non-linear techniques"

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. t-SNE and UMAP reveal category grouping: alt.atheism and soc.religion.christian each concentrate in different regions, and sci.med forms several tight, isolated islands, with UMAP showing the clearest regional separation. However, the separation is incomplete in both, and some overlap remains in the center, mainly between comp.graphics and sci.med. PCA shows almost complete overlap: most documents are packed near the origin with all four categories mixed together, and only a few comp.graphics documents are pushed far away, stretching the axes. The three filtering methods combined with pattern features show the same trend.

2. There are two reasons. (a) PCA is a linear projection that can only "flatten" the data from a single angle, so it cannot unfold the curved, interleaved category structure in high-dimensional space; t-SNE and UMAP are non-linear methods that preserve neighborhood

relationships. (b) PCA maximizes variance, not class separability. In an unnormalized term-frequency matrix, the variance is dominated by a few documents with extreme term counts (the few comp.graphics points pushed far away in the plot), so the directions PCA finds mainly separate these documents from the rest and do little to separate the four categories.

3. t-SNE aims only to preserve local neighborhood relationships, not the relative distances between clusters. It uses a heavy-tailed Student-t distribution in the low-dimensional space, which deliberately pushes different clusters apart and exaggerates the gaps between them; cluster sizes are also rescaled and lose their meaning. In addition, the notebook does not set random_state, so cluster positions change with every run, which shows that inter-cluster distance is not stable information and cannot be used to judge how similar two categories are.

AI used for this answer: Yes

If yes, how: After completing my answers myself, I used AI to translate them into English and polish the grammar and wording.

External references (links):

- Course slides, Yi-Shin Chen, *Data Mining*, "Session 1 and 2 – Introduction Handout", p. 44 "Dimensionality Reduction: PCA": "Goal is to find a projection that captures the largest amount of variation in data … Find the eigenvectors of the covariance matrix"
- Same handout, p. 43 "Dimensionality Reduction": "Techniques: Linear: Principle Component Analysis (PCA) … Others: supervised and non-linear techniques"
- Lab slides, "DM2026 Lab 1 Master Notebook slides", p. 26 "5.5 Dimensionality Reduction": "PCA: Linear projection that maximizes global variance. t-SNE: Non-linear technique preserving local neighborhood probabilities. UMAP: Manifold learning preserving both local clusters and broader global layout."
- M. Wattenberg, F. Viégas, I. Johnson, "How to Use t-SNE Effectively," *Distill*, 2016: "Cluster sizes in a t-SNE plot mean nothing"; "Distances between clusters might not mean anything" — https://distill.pub/2016/misread-tsne/
- scikit-learn documentation, sklearn.manifold.TSNE — https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html

---

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. Four columns are created, one for each category.

2. The category is a nominal attribute, which only supports checking whether two values are the same or different; the integers 0–3 are merely codes assigned by the alphabetical order of the category names. If they are used directly as numeric input, the algorithm wrongly assumes that the categories are ordered (soc.religion.christian = 3 > sci.med = 2) and that the distances between them are meaningful and equally spaced. For example, KNN would compute a distance of $|0 - 1| = 1$ between alt.atheism and comp.graphics but $|0 - 3| = 3$ between alt.atheism and soc.religion.christian, as if atheism were closer to computer graphics than to Christianity. Linear regression can assign only one coefficient to this column, forcing the category effects to change monotonically at equal intervals (0w, 1w, 2w, 3w).

3. One-hot encoding gives each category its own independent dimension, so the four vectors are orthogonal and have no order. The Euclidean distance between any two different categories is always $\sqrt{2}$, so no artificial "closer pair" exists. A linear model can also learn an independent coefficient for each category, free from the equal-spacing constraint.

AI used for this answer: Yes

If yes, how: After completing my answers myself, I used AI to translate them into English and polish the grammar and wording.

External references (links):

- Course slides, Yi-Shin Chen, *Data Mining*, "Session 1 and 2 – Introduction Handout", p. 12 "Types of Attributes": "Nominal: Categorical (=, ≠) … Ordinal: Rank Order (<, >) … Interval: Equal Spacing (+, −) … Ratio: Proportional (×, ÷)"
- Same handout, p. 13 "Nominal": "Nominal values can only distinguish one object from another"
- Same handout, p. 36 "Data Preprocessing": "Discretization and binarization"
- scikit-learn documentation, sklearn.preprocessing.LabelBinarizer — https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html

---